

大规模高可用多集群架构

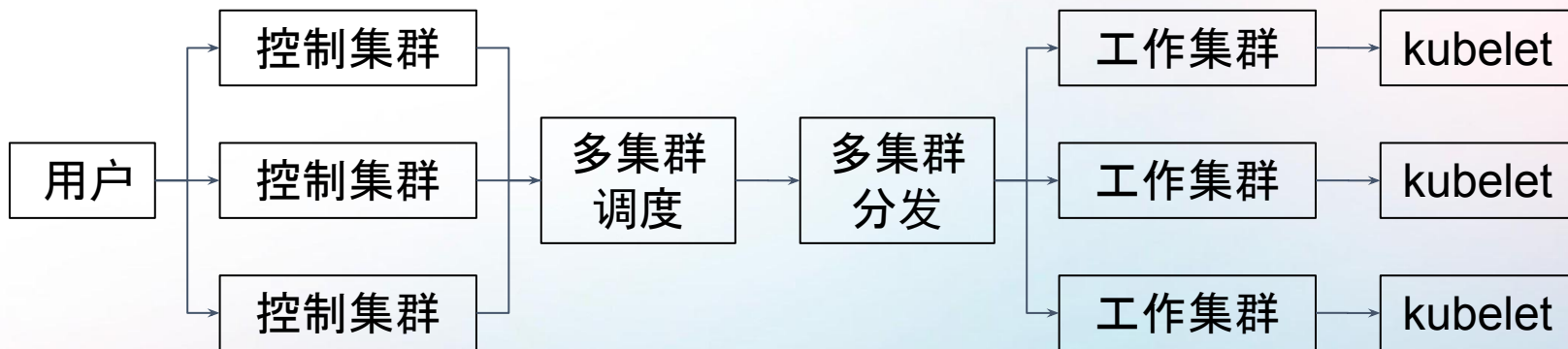
陈君彦

字节跳动软件工程师

2025/11/15



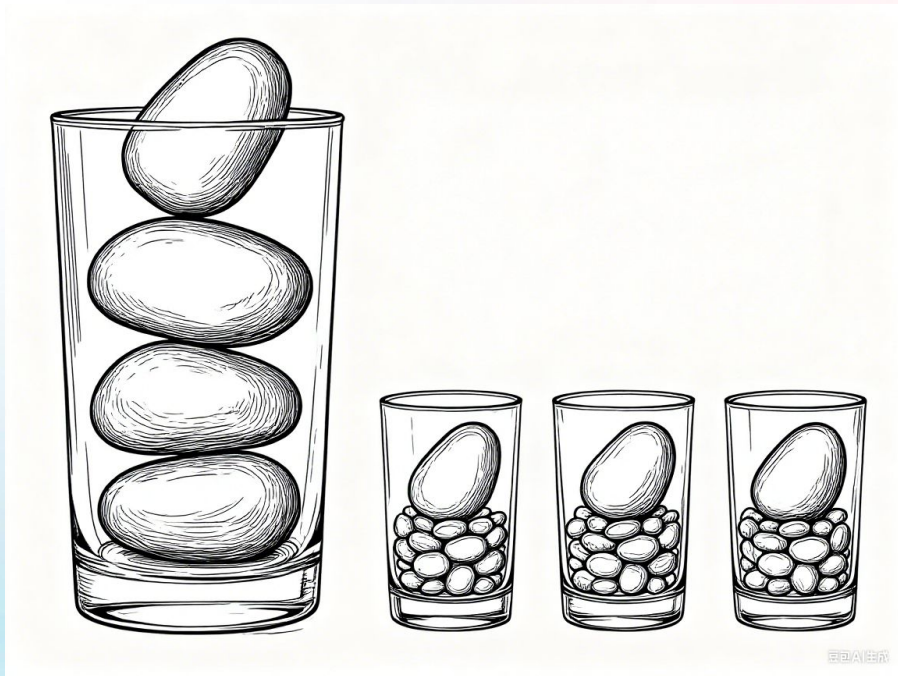
什么是多集群？



为什么要多集群？

为什么要多集群？

1. 可扩展性
2. 资源弹性
3. 故障隔离



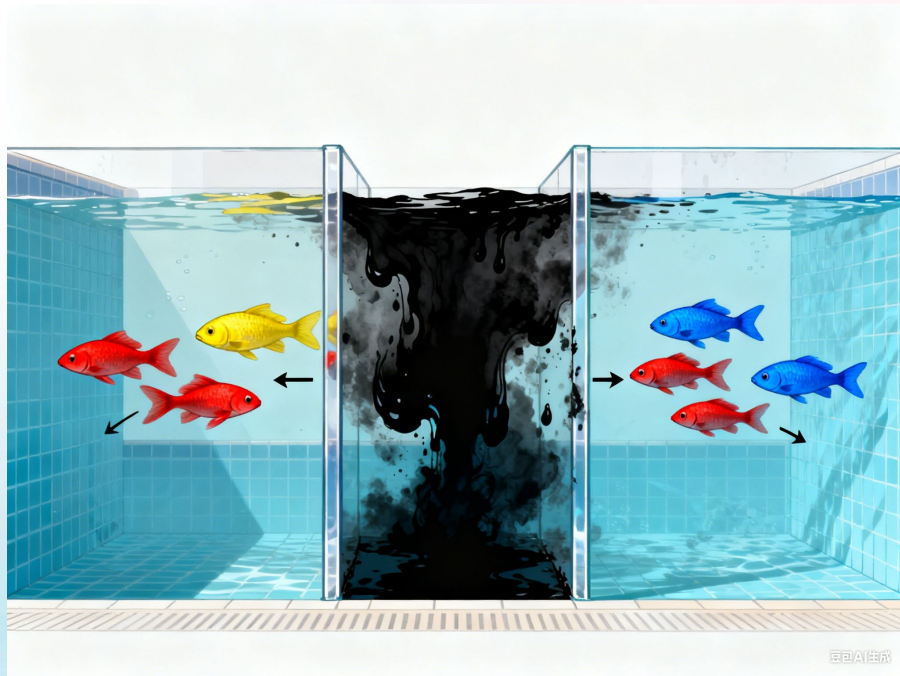
为什么要多集群？

1. 可扩展性
2. 资源弹性
3. 故障隔离



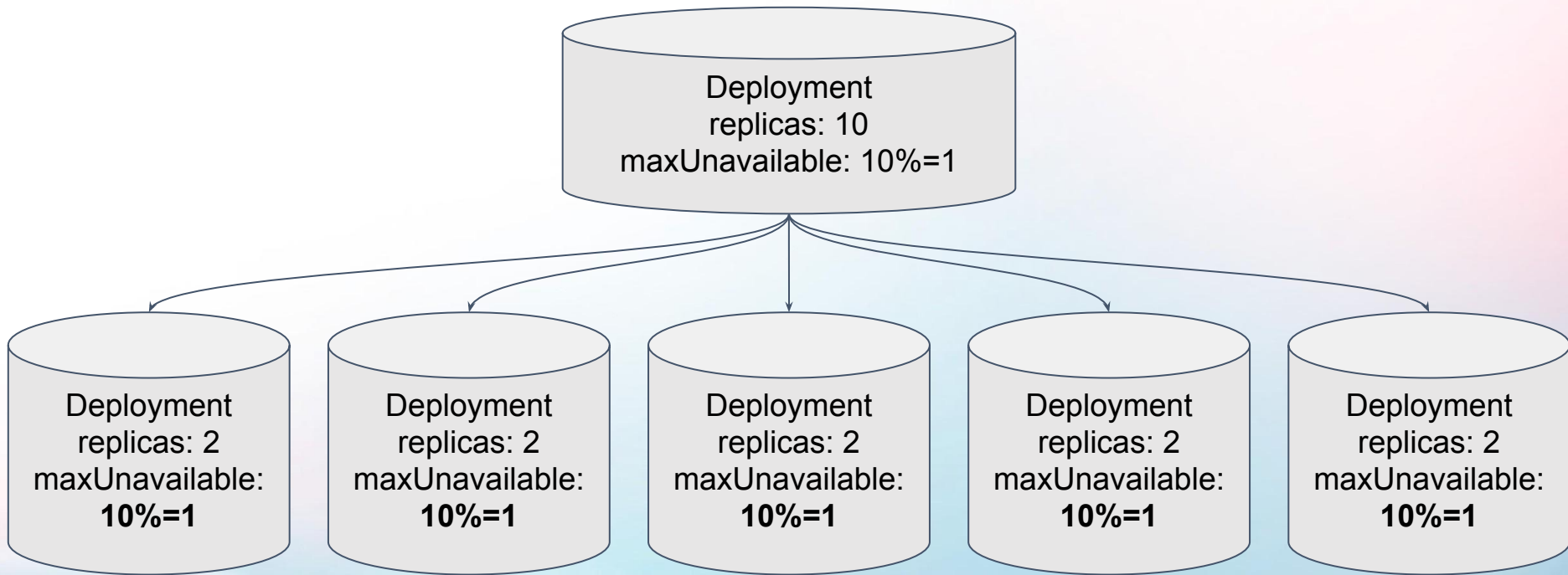
为什么要多集群？

1. 可扩展性
2. 资源弹性
3. 故障隔离



多集群的挑战

发布滚动粒度



maxUnavailable 总和: 5

资源重均衡

A: 30

B: 30

C: 30

D: 30

maxUnavailable: 10

A: 0

B: 40

C: 40

D: 40

资源重均衡

A: 30

B: 30

C: 30

D: 30

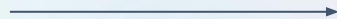


A: 0

B: 30

C: 30

D: 30



A: 0

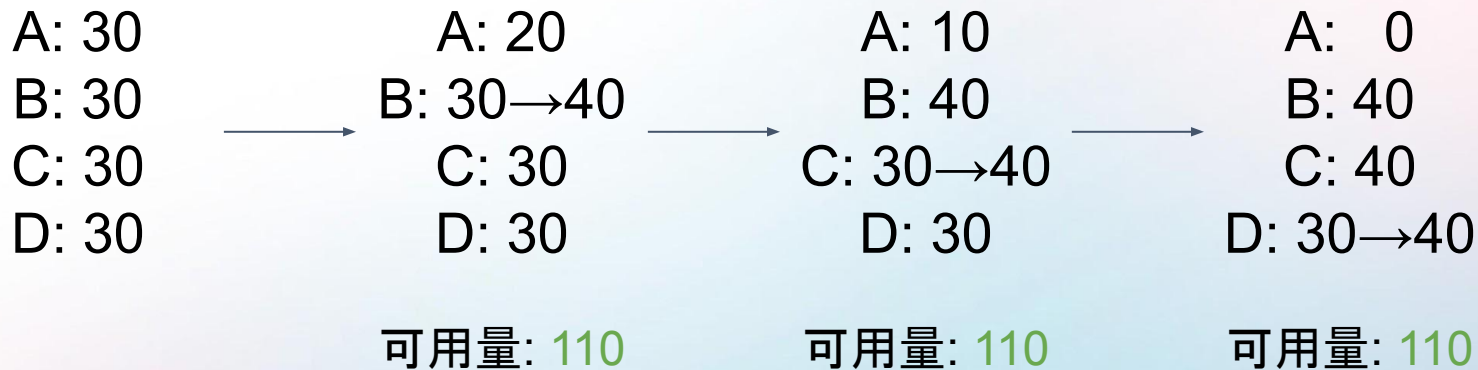
B: 40

C: 40

D: 40

可用量: 90

资源重均衡



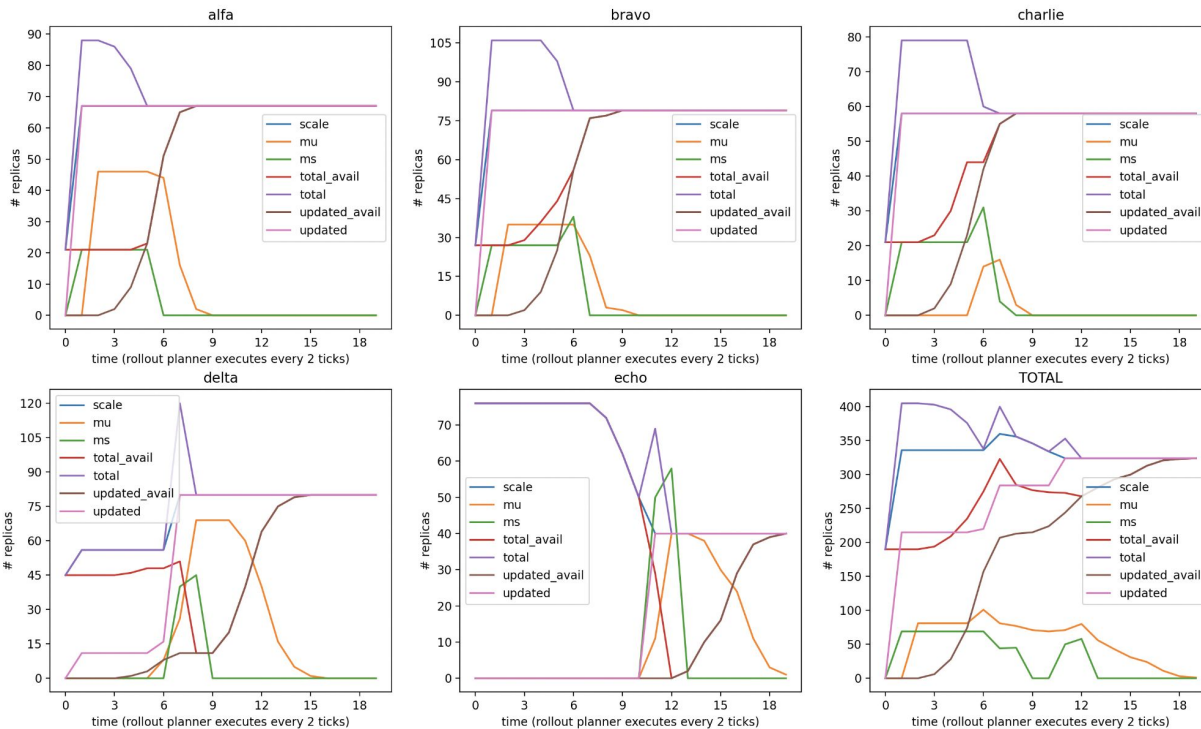
多集群协调控制循环

给定当前副本分布 x 和目标分布 y

求中间状态 x' 满足以下限制条件：

- 全局约束 (如 maxUnavailable、maxSurge)
- 最大化滚动吞吐量
- 稳定、确定性的行为

效果

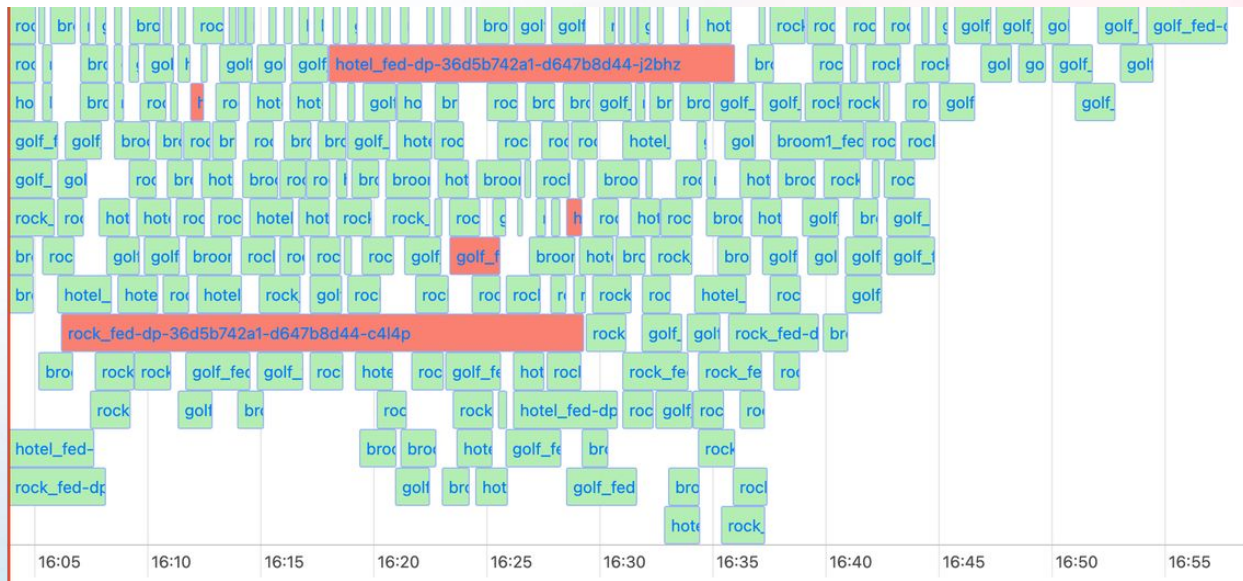


更复杂的工作负载

- 其他工作负载类型
 - 容器 exec/restart
 - 批量发布 (Batched rollout)
 - 基于时间的自动扩缩容

收益

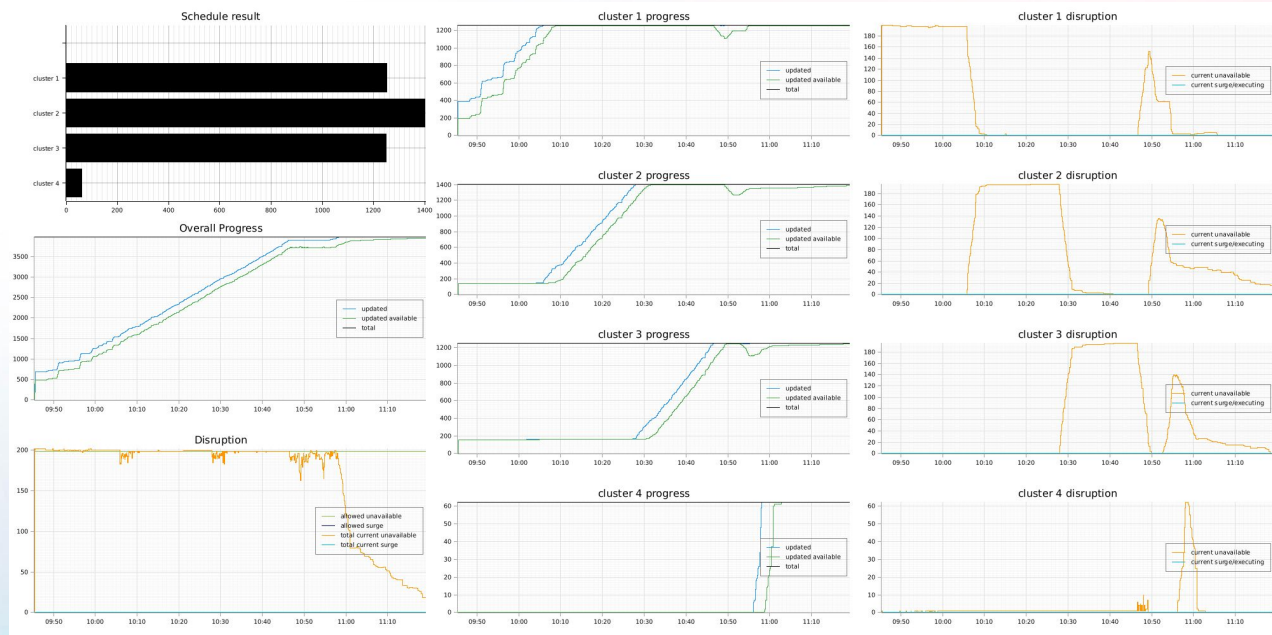
- 精准滚动粒度
- 提升长尾效率
- 提升滚动稳定性
- 行为更可预测



启用全局编排前，升级期间每个实例的滚动时长
后期并行执行的pod数不断变小

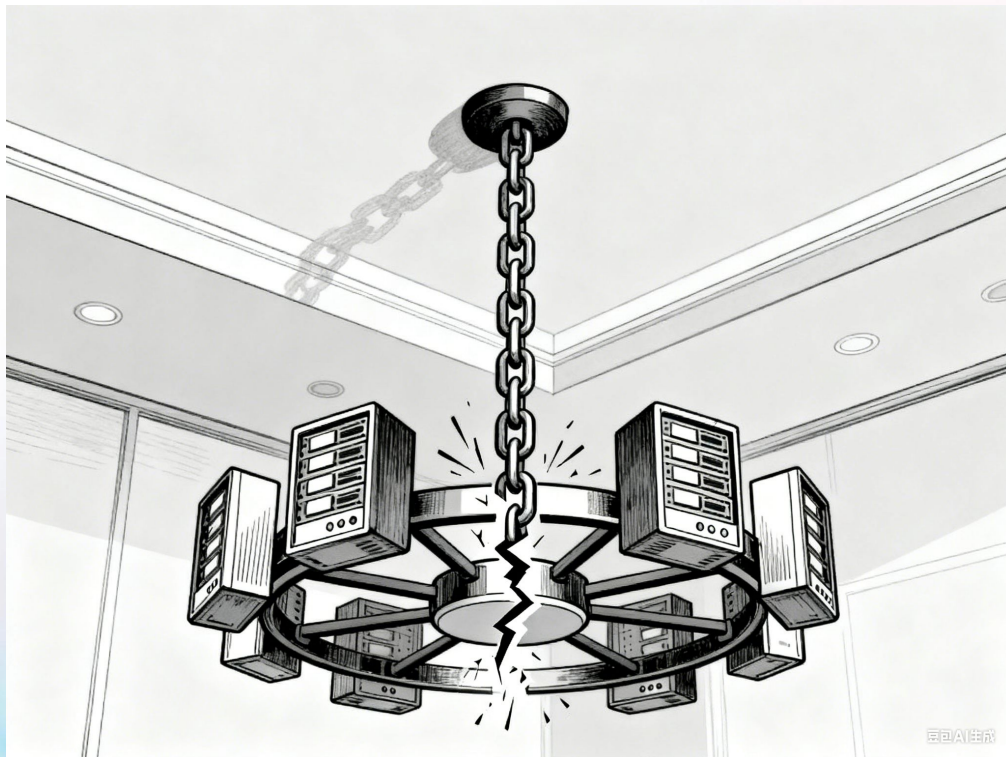
收益

- 精准滚动粒度
- 提升长尾效率
- 提升滚动稳定性
- 行为更可预测



4号集群升级完成后归还 PDB 配额, 均分到所有集群
驱逐(右边橙线第二轮波峰)只在 10:45 后才发生

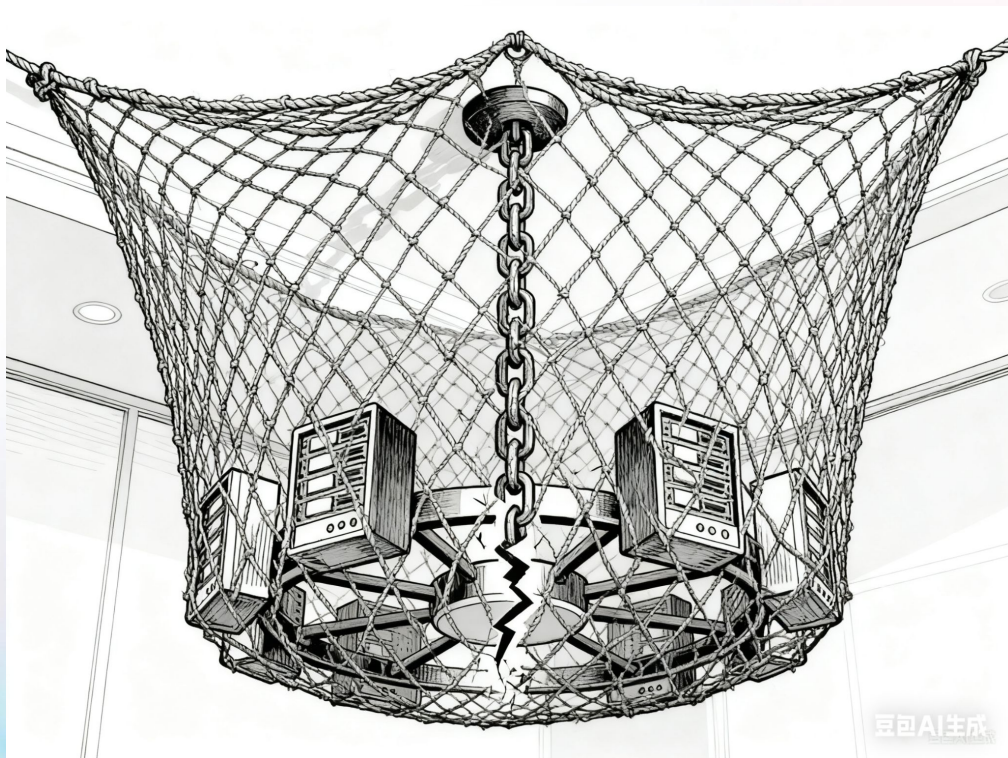
控制面集中的风险



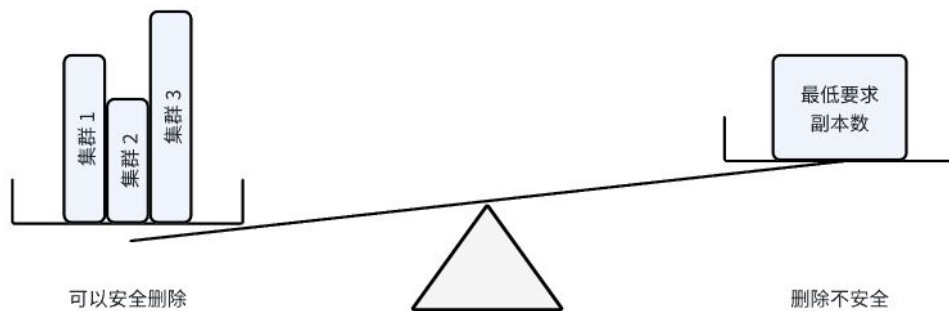
集群自发操作协调

- 调度器重调度
- 单机驱逐
- 节点维护 drain 操作

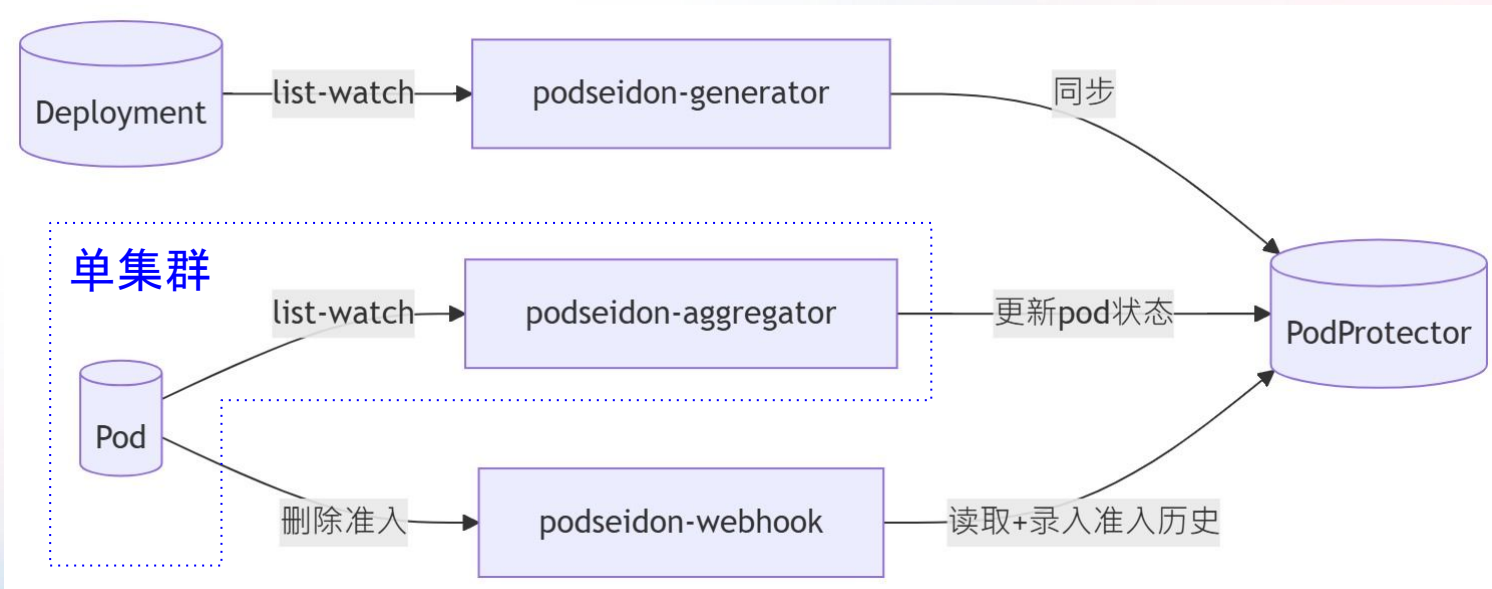
全局端到端保护



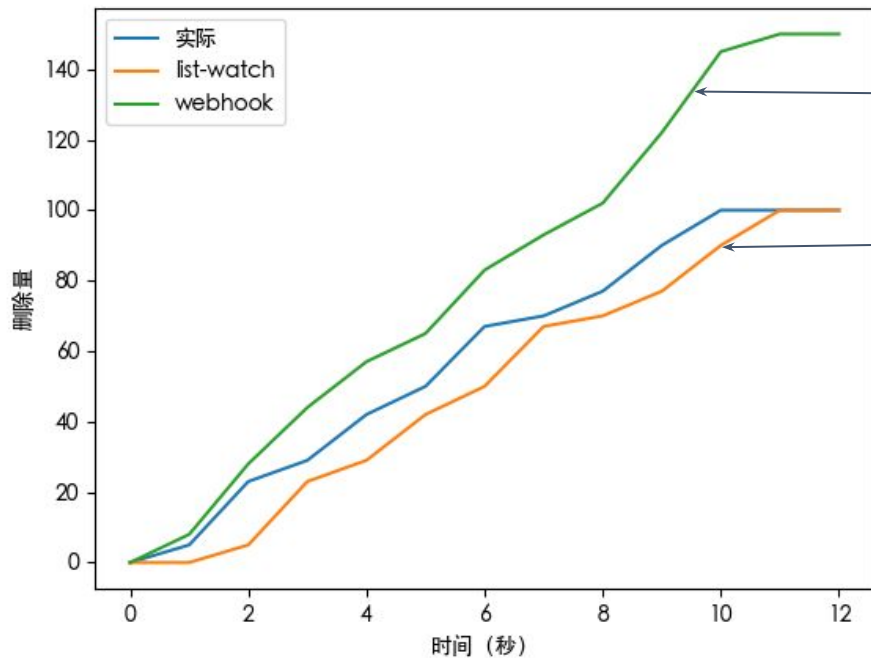
全局端到端保护



架构



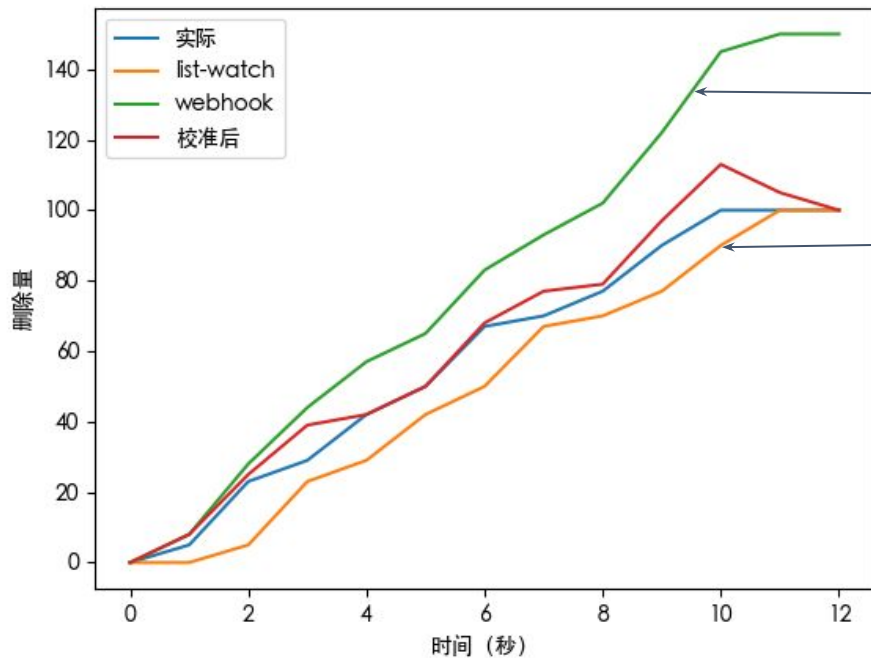
可用性估算



webhook 无限向上发散

list-watch (aggregator)
持续向下偏离

可用性估算

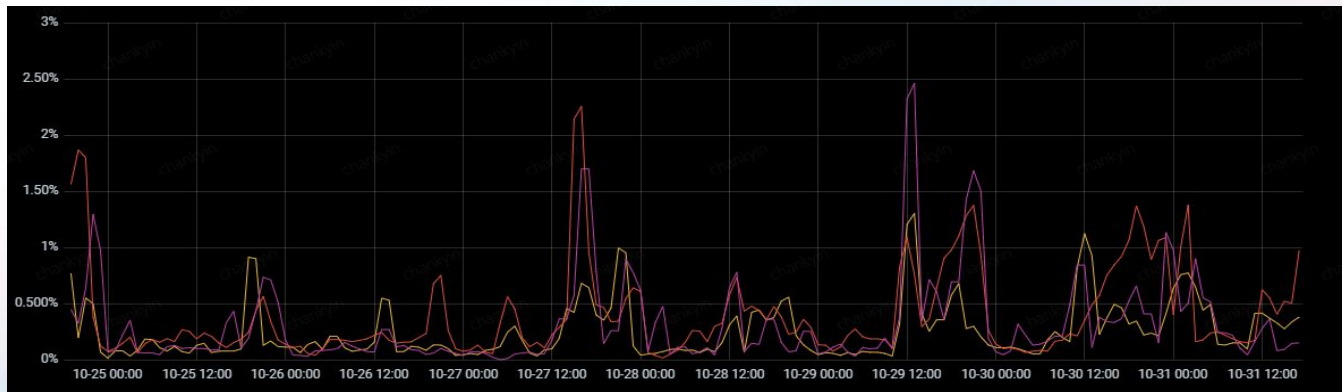


webhook 无限向上发散

list-watch (aggregator)
持续向下偏离

成效

- 可能存在竞态
- 时钟偏差



7 天内每小时 webhook 拦截率
(含真陽性及假陽性)

容灾能力

- etcd 数据损坏
- 控制器 bug

可扩展性

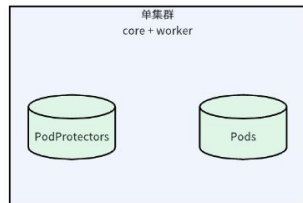
- 优化点：
 - 压缩事件, 限制对象大小
 - 动态 label selector 索引
 - Webhook 请求批处理
 - Pod Watch 分片

可扩展性

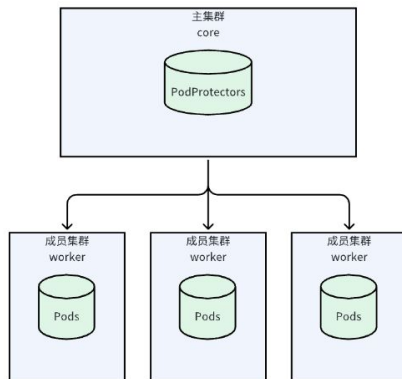
- 最大机房常态峰值
 - 23 个微服务集群
 - 16 万个 PodProtector
 - 150 万个受保护 Pod
 - 7 万 事件/秒
 - 400 首次删除/秒

各种多集群架构

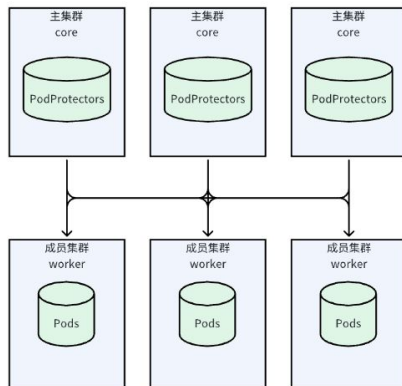
单集群部署



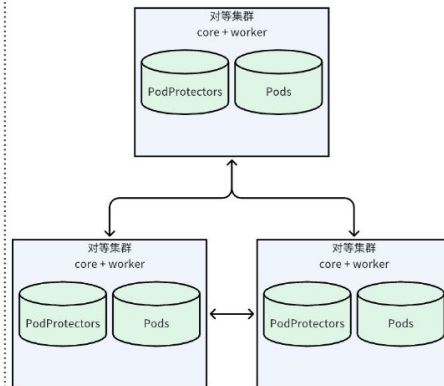
单主多成员联邦架构



分布式多主多成员联邦架构



分布式集群对等架构



总结

- Podseidon: 自下而上的全局可用性安全网
 - <https://github.com/kubewharf/podseidon>
- 滚动编排: 自上而下的多集群发布协调

Group: 字节跳动云原生与开源交流2 

